
Exception Handling

Dr. Harish D. Gadade (डॉ. हरिष डी. गडदे)
Dept. of Computer Science and Engg.
COEP Technological University, Pune

Exception Handling

- What is Exception?
- What is Exception Handling
- Binding Exception to Handler
- Design Issues

What is Exception?

In programming, an **exception** is an **unexpected event or error** that occurs while a program is running, which disrupts the normal flow of execution.

Example:

- `Powercutexception`
- `illEcxception`
- `FileNotFoundException`

What is Exception Handling?

- The Alternate source through which we can handle the exception.
- Exception handling is a mechanism in programming that allows developers to detect, handle, and recover from runtime errors (exceptions) without crashing the entire program.

Example:

- **Powercutexception**
- **illEcxeption**
- **FileNotFoundException**

In Programming

1. **Cause:** Exceptions usually happen due to invalid operations, such as:
 - Dividing by zero
 - Accessing an invalid index in a list/array
 - Opening a file that doesn't exist
 - Network/IO failures
2. **What happens:**
 - When an exception occurs, the program stops executing its normal sequence.
 - The control is transferred to a special part of the program called an **exception handler**.

Binding Exceptions to Handler

When we talk about **binding exceptions to a handler**, we mean **associating specific exception types with specific handling code** so that when those exceptions occur, the corresponding handler is executed.

- Exceptions are "raised" (or "thrown") when an error occurs.
- Handlers are code blocks that "catch" and deal with exceptions.
- Binding means deciding **which handler will catch which exception**.

◆ General Exception Handling Model

Most programming languages (Java, Python, C++, etc.) use these keywords:

1. **try** – Block of code where exception might occur
2. **catch / except** – Block of code to handle the exception
3. **finally** – (Optional) Block that always executes (cleanup code)
4. **throw / raise** – Used to generate an exception manually

Exception Handling in C

C Program to check and rectify divide by zero condition

```
#include <stdio.h>
#include <stdlib.h>
void function(int);
int main()
{
    int x = 0;
    function(x);
    return 0;
}
```

```
void function(int x)
{
    float fx;
    if (x == 0) {
        printf("Error: Division by Zero Error\n");
        exit(EXIT_FAILURE);
    }
    else {
        fx = 10 / x;
        printf("f(x) is: %.5f", fx);
    }
}
```

- **C differs fundamentally** from languages like Java, Python, or C++ because **C does not have built-in exception handling**.

Exception Handling in Python

Exception handling in Python is done using `try-except` blocks.

Basic Structure

```
try:  
    # Code that may raise an exception  
except ExceptionType:  
    # Code to handle that specific exception
```

Example 1: Divide by Zero

```
try:  
    x = 0  
    result = 10 / x  
    print("Result:", result)  
except ZeroDivisionError:  
    print("✗ Error: Division by zero is not allowed.")
```

Exception Handling in Python

Example 2: Multiple Exceptions

```
try:
    num = int(input("Enter a number: "))
    result = 10 / num
    print("Result:", result)
except ZeroDivisionError:
    print("✗ You cannot divide by zero.")
except ValueError:
    print("✗ Invalid input! Please enter a number.")
```

Here,

- ***ZeroDivisionError*** is bound to the first Handler
- ***ValueError*** is bounded to the second Handler

Exception Handling in Python

Example 3: Using `finally`

```
try:
    file = open("data.txt", "r")
    content = file.read()
    print(content)
except FileNotFoundError:
    print("✗ File not found.")
finally:
    print("✓ This block always executes (cleanup).")
```

Output

✗ File not found.
✓ This block always executes (cleanup).

=== Code Execution Successful ===

Exception Handling in Python

Example 3: Using `finally`

```
try:
    file = open("data.txt", "r")
    content = file.read()
    print(content)
except FileNotFoundError:
    print("✗ File not found.")
finally:
    print("✓ This block always executes (cleanup).")
```

Output

✗ File not found.
✓ This block always executes (cleanup).

=== Code Execution Successful ===

✓ Summary:

- `try` → risky code
- `except` → handle error
- `finally` → always runs (cleanup)

Activate Windows
Go to Settings to activate Windows

Exception Handling in Python

Python Program to check and rectify divide by zero condition

```
try:
    num = int(input("Enter a number: "))
    result = 10 / num
    print("Result:", result)
except ZeroDivisionError:
    print("✗ Error: Division by zero is not allowed.")
except ValueError:
    print("✗ Error: Invalid input. Please enter a number.")
finally:
    print("✅ Program ended (with or without error).")
```

Design Issues for Exception Handling

1. How Are Exceptions Bound to Handlers?
2. How Are Exceptions Raised?
3. What Happens After an Exception Is Handled?
4. How Do Exceptions Propagate?
5. Granularity of Exceptions
6. Checked vs. Unchecked Exceptions

Design Issues for Exception Handling

1. How Are Exceptions Bound to Handlers?

- **Static binding:** Handlers determined at compile time.
- **Dynamic binding:** Handler chosen at runtime by matching exception type.
- **Question:** Should binding be static or dynamic?

When an exception is raised:

1. The runtime system **creates the exception object** (with type + message + stack info).
2. It begins a **search for a matching handler**:
 - Starts in the **nearest enclosing try block**.
 - If no handler matches, the runtime **unwinds the stack** (pops function frames).
 - At each level, it looks for a handler that matches the exception's type.
3. Once a **suitable handler** is found, the exception is **bound** (associated) to it, and control is transferred there.

Design Issues for Exception Handling

2. How Are Exceptions Raised?

- Explicitly (via `throw`, `raise`) by the programmer.
- Implicitly (by runtime system) for errors like divide-by-zero, null pointer access.
- Question: Should the language support both implicit and explicit raising?

Example of Explicit

```
def divide(a, b):  
    if b == 0:  
        raise ZeroDivisionError("Cannot divide by zero")  
    return a / b
```

Example of Implicit

```
x=1/0 # Runtime automatically raises ZeroDivisionError  
print(x)
```

Design Issues for Exception Handling

3. What Happen After an Exception is Handled?

1. Normal Execution Resumes (Same Scope)

- In some languages (Python, Java, C++), after the `catch` / `except` block executes, control resumes with the **first statement after the try-catch block**.
- Example (Python):

```
try:
    x = 10 / 0
except ZeroDivisionError:
    print("Handled division by zero")
print("Program continues here") # resumes normal execution
```

Design Issues for Exception Handling

3. What Happen After an Exception is Handled?

2. Finally / Cleanup Runs

- If a `finally` block exists, it **always runs** (whether or not the exception was caught).
- Ensures resources (files, sockets, memory) are cleaned up.
- Example (Java):

```
try:
    file = open("data.txt", "r")
    content = file.read()
    print(content)
except FileNotFoundError:
    print("✗ File not found.")
finally:
    print("✓ This block always executes (cleanup).")
```

Design Issues for Exception Handling

4. How do Exception Propagates?

When an exception is raised but **not handled in the current scope**, the runtime system looks for a matching handler in the **call stack**. If it's not found locally, the exception is **propagated upward** (function → caller → caller's caller, etc.) until:

- a handler is found, OR
- the program reaches the top-level without handling → program terminates.

This process is also called **stack unwinding**.

Design Issues for Exception Handling

4. How do Exception Propagates?

```
def f1():  
    f2()  
  
def f2():  
    f3()  
  
def f3():  
    raise ValueError("Something went wrong")  
  
try:  
    f1()  
except ValueError as e:  
    print("Caught exception:", e)
```

- `f3()` raises → no handler there → propagates to `f2()` → no handler → propagates to `f1()` → no handler → propagates to `try` in main → caught.

Design Issues for Exception Handling

5. Granularity of Exceptions

- Should the language define only a few general exceptions (like in C with error codes)?
- Or a rich hierarchy of specific exception classes (Java, Python)?
- **Question:** How detailed should exception classification be?

Granularity of exceptions refers to how **fine-grained** or **coarse-grained** the exception **types** are in a programming language or system.

It's about deciding **how many different exception classes** you create, and **how specific** each exception is.

- **Fine-Grained Exceptions** → many small, specific exception classes.
- **Coarse-Grained Exceptions** → fewer, broader exception classes.

Design Issues for Exception Handling

5. Granularity of Exceptions

◆ Fine-Grained Exceptions

- Each specific error type gets its **own exception class**.
- Allows **precise error handling** (catch exactly what you want).

```
try {  
    readFile();  
} catch (FileNotFoundException e) {  
    // handle missing file  
} catch (EOFException e) {  
    // handle premature end-of-file  
} catch (IOException e) {  
    // handle other I/O problems  
}
```

◆ Coarse-Grained Exceptions

- Fewer exception types cover **broad categories of errors**.
- Easier to implement, but less control over specific cases.

```
try:  
    read_file()  
except IOError:  
    print("Some file error occurred")
```

Design Issues for Exception Handling

6. Checked vs. Unchecked Exceptions

- **Checked:** Must be declared and handled (Java).
- **Unchecked:** No requirement to declare/handle (C++, Python).
- **Question:** Should exceptions be enforced at compile-time or left flexible?

1. Checked Exceptions

- **Definition:** Exceptions that the **compiler forces you to handle** (either catch them or declare them with `throws`).
- **Examples:**
 - `IOException`
 - `SQLException`
 - `ClassNotFoundException`

2. Unchecked Exceptions

- **Definition:** Exceptions that the compiler does **not require you to handle**.
- **Examples:**
 - `NullPointerException`
 - `ArrayIndexOutOfBoundsException`
 - `ArithmeticException`
 - All subclasses of `RuntimeException`